Lesson 15 - Homework

Object-Oriented Programming (OOP) - II

Mövzular: Inheritance, `super()`, Method Overriding, Polymorphism, Encapsulation, `@property`, `@staticmethod`, `@classmethod`, Composition.

Dərsdə öyrəndiyin OOP anlayışlarını yeni bir layihədə tətbiq edəcəksən. Əvvəlcə izahlı kod nümunələrini oxu, sonra təlim mərkəzinin idarəetmə sistemini özün qur.

1. Ev tapşırığının məqsədi

Bu ev tapşırığında class-ların necə əlaqələndirildiyini, eyni adlı metodun müxtəlif obyektlərdə necə işlədiyini və məlumatları necə yoxlayıb qorumağı məşq edəcəksən.

İş qaydası: Əvvəlcə **2-ci bölmədəki xatırlatmaları** oxu. Oradakı nümunələr onlayn mağaza sistemi üzərində qurulub. **3-cü və 4-cü bölmələrdəki tapşırıqlar** isə fərqli layihədir: **Təlim Mərkəzinin İdarəetmə Sistemi**. Tapşırıqları ardıcıllıqla həll et və hazır nümunələri olduğu kimi köçürmə.

---

2. Mövzuların xatırlatması

Bu bölmədə hər anlayış üçün işlək Python kodu, gözlənilən nəticə və həmin kodla birbaşa əlaqəli izah verilib. **Hər nümunəni ayrıca işlət:** onlar mövzunu müstəqil göstərmək üçün yazılıb, hamısını eyni Python faylına ardıcıl köçürmək tələb olunmur.

2.1. Inheritance (Mirasalma)

Inheritance nədir? Bir class-ın başqa class-da müəyyən edilmiş atribut və metodlardan istifadə edə bilməsidir.

Mağazada köynək də, dəftər də məhsuldur. Hər ikisinin adı, qiyməti və stok sayı ola bilər. Bu ortaq məlumatları hər məhsul növündə təkrar yazmamaq üçün `Mehsul` class-ından miras alan `Geyim` class-ı yaradırıq.

```
class Mehsul:
    def __init__(self, ad, qiymet, stok):
        self.ad = ad
        self.qiymet = qiymet
        self.stok = stok

    def melumat_goster(self):
        return f"{self.ad} | {self.qiymet} AZN | Stok: {self.stok}"


class Geyim(Mehsul):
    pass


koynek = Geyim("Köynək", 50, 20)
print(koynek.melumat_goster())
```

Nəticə:

```
Köynək | 50 AZN | Stok: 20
```

Kodun izahı:

1. `Mehsul` əsas, yəni **parent class**-dır. Onun `__init__()` metodu ad, qiymət və stoku saxlayır.
2. `Geyim(Mehsul)` yazaraq **child class** olan `Geyim`-i yaratdıq.
3. `pass` əlavə kod yazmadığımızı göstərir. Buna baxmayaraq, `Geyim` parent class-ın constructor və `melumat_goster()` metodundan istifadə edir.
4. `koynek.melumat_goster()` məhz bu səbəbdən işləyir.

Yadda saxla: `class YeniClass(EsasClass):` yazılışında mötərizədə miras alınan class göstərilir.

2.2. `super()`

`super()` nədir? Child class daxilindən parent class-ın metoduna müraciət etməyin üsuludur.

Əvvəlki `Geyim` class-ına `olcu` adlı əlavə atribut daxil etmək istəyirik. Ad, qiymət və stoku yenidən yazmamaq üçün parent constructor-u `super()` ilə çağırırıq.

```
class Mehsul:
    def __init__(self, ad, qiymet, stok):
        self.ad = ad
        self.qiymet = qiymet
        self.stok = stok


class Geyim(Mehsul):
    def __init__(self, ad, qiymet, stok, olcu):
        super().__init__(ad, qiymet, stok)
        self.olcu = olcu


koynek = Geyim("Köynək", 50, 20, "M")
print(koynek.ad)
print(koynek.qiymet)
print(koynek.stok)
print(koynek.olcu)
```

Nəticə:

```
Köynək
50
20
M
```

Kodun izahı:

1. `Geyim` constructor-u dörd parametr qəbul edir: `ad`, `qiymet`, `stok`, `olcu`.
2. `super().__init__(ad, qiymet, stok)` parent class-da ilk üç atributu yaradır.
3. `self.olcu = olcu` isə yalnız geyimə aid olan ölçünü saxlayır.
4. Beləliklə, ortaq constructor kodunu təkrar yazmırıq.

Vacib: Child class-da ayrıca `__init__()` yaradanda parent constructor avtomatik çağırılmır. `super()` başqa parent metodları üçün də istifadə oluna bilər.

2.3. Method Overriding

Method Overriding nədir? Child class-da parent class-dakı metodla eyni adda metod yaradaraq davranışını dəyişməkdir.

`Mehsul` ad və qiyməti göstərir, `Geyim` isə bunlara əlavə olaraq ölçünü göstərməlidir. Bunun üçün `melumat_goster()` metodunu child class-da yenidən yazırıq.

```
class Mehsul:
    def __init__(self, ad, qiymet, stok):
        self.ad = ad
        self.qiymet = qiymet
        self.stok = stok

    def melumat_goster(self):
        return f"{self.ad} | {self.qiymet} AZN"


class Geyim(Mehsul):
    def __init__(self, ad, qiymet, stok, olcu):
        super().__init__(ad, qiymet, stok)
        self.olcu = olcu

    def melumat_goster(self):
        esas_melumat = super().melumat_goster()
        return f"{esas_melumat} | Ölçü: {self.olcu}"


mehsul = Mehsul("Dəftər", 5, 20)
koynek = Geyim("Köynək", 50, 10, "M")
print(mehsul.melumat_goster())
print(koynek.melumat_goster())
```

Nəticə:

```
Dəftər | 5 AZN
Köynək | 50 AZN | Ölçü: M
```

Kodun izahı:

1. Hər iki class-da eyni adlı `melumat_goster()` metodu var.
2. `Geyim` daxilində `super().melumat_goster()` parent metodunun qaytardığı mətni alır.
3. Həmin mətnə `self.olcu` əlavə edilir və yeni nəticə `return` olunur.
4. Əsas `Mehsul` class-ının davranışı dəyişmir, yalnız `Geyim` üçün genişlənir.

Yadda saxla: Eyni metod adı, child class-da dəyişdirilmiş davranış.

2.4. Polymorphism

Polymorphism nədir? Eyni metod çağırışının fərqli obyektlərdə müxtəlif nəticə verməsidir.

Mağazadakı `Geyim` məhsulunun ölçüsü, `Qida` məhsulunun isə son istifadə tarixi var. Hər ikisində eyni `melumat_goster()` metodunu çağırsaq da, nəticə fərqli olacaq.

```
class Mehsul:
    def __init__(self, ad, qiymet):
        self.ad = ad
        self.qiymet = qiymet

    def melumat_goster(self):
        return f"{self.ad} | {self.qiymet} AZN"


class Geyim(Mehsul):
    def __init__(self, ad, qiymet, olcu):
        super().__init__(ad, qiymet)
        self.olcu = olcu

    def melumat_goster(self):
        return f"{super().melumat_goster()} | Ölçü: {self.olcu}"


class Qida(Mehsul):
    def __init__(self, ad, qiymet, son_tarix):
        super().__init__(ad, qiymet)
        self.son_tarix = son_tarix

    def melumat_goster(self):
        return f"{super().melumat_goster()} | Son tarix: {self.son_tarix}"


mehsullar = [
    Geyim("Köynək", 50, "M"),
    Qida("Süd", 3, "10.11.2026"),
]

for mehsul in mehsullar:
    print(mehsul.melumat_goster())
```

Nəticə:

```
Köynək | 50 AZN | Ölçü: M
Süd | 3 AZN | Son tarix: 10.11.2026
```

Kodun izahı:

1. `Geyim` və `Qida` `Mehsul`-dan miras alır, amma öz əlavə atributları var.
2. Hər ikisi `melumat_goster()` metodunu özünə uyğun dəyişir.
3. Obyektlər eyni list-də saxlanılır.
4. `for` dövründə tək bir metod çağırışı (`mehsul.melumat_goster()`) edilir. Python uyğun obyektin metodunu işlədir.

Vacib: Dövr daxilində obyektin geyim və ya qida olduğunu `if/elif` ilə yoxlamaq lazım deyil.

2.5. Encapsulation və `@property`

Encapsulation nədir? Obyektin məlumatlarını və onlara aid davranışları birlikdə saxlamaq, məlumatların necə dəyişdirildiyini qaydalarla idarə etməkdir.

Mağazada stok mənfi ola bilməz. `@property` və setter vasitəsilə stoka yanlış dəyər təyin olunmasının qarşısını alırıq.

```
class Mehsul:
    def __init__(self, ad, stok):
        self.ad = ad
        self.stok = stok

    @property
    def stok(self):
        return self._stok

    @stok.setter
    def stok(self, yeni_stok):
        if yeni_stok < 0:
            raise ValueError("Stok mənfi ola bilməz")
        self._stok = yeni_stok


mehsul = Mehsul("Köynək", 10)
print(mehsul.stok)
mehsul.stok = 20
print(mehsul.stok)

try:
    mehsul.stok = -5
except ValueError as xeta:
    print("Xəta:", xeta)

print(mehsul.stok)
```

Nəticə:

```
10
20
Xəta: Stok mənfi ola bilməz
20
```

Kodun izahı:

1. `@property` stoku `self._stok`-dan oxuyur.
2. `@stok.setter` `mehsul.stok = ...` yazıldıqda çağırılır.
3. Mənfi dəyər üçün `ValueError` yarandığına görə `self._stok` dəyişmir.
4. Buna görə son `print` əvvəlki düzgün stok olan `20`-ni göstərir.

Vacib: Setter daxilində `self.stok = yeni_stok` yazma. Bu, setter-i yenidən çağırıb `RecursionError` yarada bilər. Daxili dəyəri `self._stok`-da saxla. `_` tam gizlilik yox, daxili istifadə üçün adlandırma qaydasıdır.

2.6. `@staticmethod`

Static method nədir? Konkret obyektin atributlarına ehtiyac duymayan, lakin class ilə əlaqəli köməkçi metoddur.

Məhsulun stok sayının mənfi olub-olmadığını hər hansı məhsul obyekti yaratmadan yoxlaya bilərik.

```
class Mehsul:
    @staticmethod
    def stok_duzgundur(say):
        return say >= 0


print(Mehsul.stok_duzgundur(10))
print(Mehsul.stok_duzgundur(0))
print(Mehsul.stok_duzgundur(-5))
```

Nəticə:

```
True
True
False
```

Kodun izahı:

1. `stok_duzgundur(say)` yalnız bir ədəd qəbul edir; nə `self`, nə də `cls` istifadə etmir.
2. `say >= 0` şərti doğru olduqda `True`, əks halda `False` qaytarır.
3. Metodu birbaşa `Mehsul.stok_duzgundur(...)` şəklində çağırırıq.

Yadda saxla: Static method obyekt yaratmadan işləyə bilən köməkçi yoxlamalar üçün əlverişlidir.

2.7. `@classmethod`

Class method nədir? İlk parametr kimi `cls` qəbul edən və class-ın özü ilə işləyən metoddur. Onun istifadələrindən biri obyekt yaratmaq üçün alternativ yol verməkdir.

Mağazaya məhsul haqqında məlumat dictionary şəklində gəlsə, ondan yeni `Mehsul` obyekti yarada bilərik.

```
class Mehsul:
    def __init__(self, ad, qiymet, stok):
        self.ad = ad
        self.qiymet = qiymet
        self.stok = stok

    @classmethod
    def sozlukden_yarat(cls, melumat):
        ad = melumat["ad"]
        qiymet = melumat["qiymet"]
        stok = melumat["stok"]
        return cls(ad, qiymet, stok)


melumat = {"ad": "Köynək", "qiymet": 50, "stok": 10}
mehsul = Mehsul.sozlukden_yarat(melumat)

print(mehsul.ad)
print(mehsul.qiymet)
print(mehsul.stok)
```

Nəticə:

```
Köynək
50
10
```

Kodun izahı:

1. Normalda `Mehsul(ad, qiymet, stok)` ilə obyekt yaradılır.
2. `sozlukden_yarat()` dictionary-dən bu üç məlumatı oxuyur.
3. `cls(ad, qiymet, stok)` yeni obyekt yaradır və onu qaytarır.
4. `self` konkret obyekti, `cls` isə metodun çağırıldığı class-ı ifadə edir.

Yadda saxla: `@classmethod` obyekt yaradılmasının fərqli üsullarını təşkil etmək üçün faydalıdır.

2.8. Composition

Composition nədir? Bir class-ın öz daxilində başqa class-lardan yaradılmış obyektlərdən istifadə etməsidir.

Mağaza kataloqu məhsul deyil, amma kataloqun içində məhsullar ola bilər. Ona görə `Kataloq` class-ını `Mehsul`-dan miras almırıq; onun daxilində məhsul obyektləri saxlayırıq.

```
class Mehsul:
    def __init__(self, ad, qiymet):
        self.ad = ad
        self.qiymet = qiymet


class Kataloq:
    def __init__(self):
        self.mehsullar = []

    def elave_et(self, mehsul):
        self.mehsullar.append(mehsul)

    def goster(self):
        for mehsul in self.mehsullar:
            print(mehsul.ad, mehsul.qiymet)


kataloq = Kataloq()
kataloq.elave_et(Mehsul("Köynək", 50))
kataloq.elave_et(Mehsul("Şalvar", 80))
kataloq.goster()
```

Nəticə:

```
Köynək 50
Şalvar 80
```

Kodun izahı:

1. `Kataloq` constructor-u boş `self.mehsullar` list-i yaradır.
2. `elave_et()` `Mehsul` obyektini həmin list-ə yerləşdirir.
3. `goster()` siyahıdakı obyektlərin adını və qiymətini çap edir.
4. `Kataloq` bir məhsul növü deyil, məhsulları **öz daxilində saxlayan** class-dır.

Yadda saxla: Inheritance: "Geyim bir məhsuldur". Composition: "Kataloqda məhsullar var".

3. Praktiki tapşırıqlar

YENİ LAYİHƏ · TƏLİM MƏRKƏZİNİN İDARƏETMƏ SİSTEMİ
Təsəvvür et ki, təlim mərkəzində müəllimlər dərs keçir, tələbələr kurslara yazılır və onların qiymətləndirmə balları sistemdə saxlanılır. Sənin məqsədin bu sistemi OOP prinsipləri ilə mərhələli qurmaqdır.

Layihədə dörd əsas class olacaq:

- `Istifadeci` — müəllim və tələbələrin ortaq məlumatları.
- `Telebe` — tələbəyə aid əlavə məlumatlar və bal.
- `Muellim` — müəllimə aid əlavə məlumatlar və ixtisas.
- `Kurs` — kursun müəllimi və qeydiyyatdan keçmiş tələbələri.

Vacib: Bütün tapşırıqlar bir-birini davam etdirir. Eyni Python faylında əvvəlki class-ları genişləndirərək işləyəcəksən.

Tapşırıq 1. Inheritance və `super()`

Məqsəd: Ortaq atributları parent class-da saxlamaq və ondan miras alan class-lar yaratmaq.

1-ci hissə: `Istifadeci` class-ı. İstifadəçinin adı (`ad`) və email ünvanı (`email`) atributlarını qəbul edən constructor yarat. `melumat_goster()` adlı metod ad və email-i bir string kimi **return** etməlidir.

Atribut	Mənası
`ad`	İstifadəçinin adı
`email`	Elektron poçt ünvanı

2-ci hissə: `Telebe` class-ı. `Istifadeci`-dən miras almalıdır. Əlavə atribut: `bal` (0–100). Constructor-da `super()` istifadə et.

3-cü hissə: `Muellim` class-ı. `Istifadeci`-dən miras almalıdır. Əlavə atribut: `ixtisas`. Constructor-da `super()` istifadə et.

Yoxlama məlumatları:

Class	Ad	Email	Əlavə məlumat
`Telebe`	Kamran	kamran@example.com	Bal: 80
`Muellim`	Elvin	elvin@example.com	İxtisas: Python Backend

Bütün atributları `print()` vasitəsilə göstər.

Gözlənilən nəticə:

```
Kamran
kamran@example.com
80
Elvin
elvin@example.com
Python Backend
```

İpucu: `super().__init__(...)` ilə parent constructor-u çağır. Yalnız child class-a məxsus əlavə atributu ayrıca saxla.

Tapşırıq 2. Method Overriding

Məqsəd: Parent metodunun nəticəsini child class-a uyğun genişləndirmək.

1. `Telebe` daxilində `melumat_goster()` metodunu yenidən yaz. Ad, email və balı qaytarsın.
2. `Muellim` daxilində həmin metodu yenidən yaz. Ad, email və ixtisası qaytarsın.
3. Hər iki metodda `super().melumat_goster()` istifadə et.

Gözlənilən nəticə:

```
Kamran | kamran@example.com | Bal: 80
Elvin | elvin@example.com | İxtisas: Python Backend
```

İpucu: Əsas mətni parent metodundan al, əlavə atributu həmin mətnə birləşdir və `return` et.

Tapşırıq 3. Polymorphism

Məqsəd: Müxtəlif class-ların obyektlərinə eyni metodla müraciət etmək.

1. İki `Telebe` və bir `Muellim` obyekti yarat.
2. Üç obyekti bir `istifadeciler` list-ində saxla.
3. `istifadecileri_goster(istifadeciler)` adlı funksiya yarat.
4. `for` dövründə hər obyektin `melumat_goster()` metodunu çağır və nəticəni göstər.

Yoxlama məlumatları:

Class	Ad	Email	Əlavə məlumat
`Telebe`	Kamran	kamran@example.com	Bal: 80
`Telebe`	Lalə	lale@example.com	Bal: 90
`Muellim`	Elvin	elvin@example.com	İxtisas: Python Backend

Gözlənilən nəticə:

```
Kamran | kamran@example.com | Bal: 80
Lalə | lale@example.com | Bal: 90
Elvin | elvin@example.com | İxtisas: Python Backend
```

Vacib şərt: Funksiyanın daxilində `if/elif` ilə obyektin tələbə və ya müəllim olduğunu yoxlama. Eyni metod çağırışından istifadə et.

Tapşırıq 4. Encapsulation və `@property`

Məqsəd: Tələbənin balını 0–100 aralığında saxlamaq.

1. `Telebe` class-ına `@property` ilə `bal` metodu əlavə et. O, `self._bal` dəyərini qaytarsın.
2. `@bal.setter` yarat. Yeni bal 0-dan az və ya 100-dən çoxdursa, `ValueError` yaratsın.
3. Bal düzgündürsə, `self._bal` daxilində saxlasın.
4. Constructor-da ilkin bal təyin olunanda da eyni setter işləsin.

Yoxlama: Kamranın ilkin balı 80 olsun. Balını 95-ə dəyiş, sonra 120 etməyə çalış. Xətanı `try/except` ilə tut və sonda balı yenidən göstər.

Gözlənilən nəticə:

```
95
Xəta: Bal 0-100 aralığında olmalıdır
95
```

İpucu: `@property`, `@bal.setter`, `self._bal` anlayışlarından istifadə et. Yanlış dəyər daxil edildikdə əvvəlki düzgün bal dəyişməməlidir.

Tapşırıq 5. `@staticmethod`

Məqsəd: Balın düzgünlüyünü yoxlayan ayrıca köməkçi metod yaratmaq.

1. `Telebe` class-ında `bal_duzgundur(bal)` adlı static method yarat.
2. 0–100 aralığındakı ballar üçün `True`, digər hallarda `False` qaytarsın.
3. Tapşırıq 4-də yazdığın setter-i yenilə: balı bu static method vasitəsilə yoxlasın.
4. `Telebe` class-ına `bal_artir(xal)` adlı instance method əlavə et. Balı verilən xal qədər artırsın. Nəticə 100-ü keçərsə, `ValueError` yaransın.

Yoxlama:

```
Telebe.bal_duzgundur(80)   → True
Telebe.bal_duzgundur(100)  → True
Telebe.bal_duzgundur(-10)  → False
Telebe.bal_duzgundur(150)  → False
```

İpucu: `bal_artir()` içərisində hesabladığın yeni balı `self.bal = ...` vasitəsilə təyin et ki, setter yoxlamanı yenidən aparsın.

Tapşırıq 6. `@classmethod`

Məqsəd: Mətn şəklində verilmiş məlumatlardan yeni istifadəçi yaratmaq.

Bəzən sistemə istifadəçi məlumatları belə daxil olur: `"Aysel;aysel@example.com"`.

1. `Istifadeci` class-ına `metnden_yarat(melumat)` adlı class method əlavə et.
2. İlk parametr `cls` olsun.
3. `split(";")` vasitəsilə mətni ada və email-ə ayır.
4. `cls(...)` ilə yeni obyekt yarat və onu `return` et.

Yoxlama məlumatları:

```
Aysel;aysel@example.com
Murad;murad@example.com
```

Gözlənilən nəticə:

```
Aysel | aysel@example.com
Murad | murad@example.com
```

Qeyd: Bu metodu tapşırıqda yalnız `Istifadeci` üzərindən çağır. `Telebe` və `Muellim` constructor-larında əlavə parametrlər var.

Tapşırıq 7. Composition — Kurs sistemi

Məqsəd: Kurs class-ında müəllim və tələbə obyektlərini istifadə etmək.

1-ci hissə: `Kurs` class-ı. Constructor `ad` və `muellim` parametrini qəbul etsin. `muellim` parametrinə bir `Muellim` obyekti veriləcək. Constructor daxilində `self.telebeler = []` yaradılmalıdır.

Atribut	Mənası
`ad`	Kursun adı
`muellim`	Kursun müəllim obyekti
`telebeler`	Kursun tələbələrinin list-i

2-ci hissə: Tələbə əlavə etmək. `telebe_elave_et(telebe)` metodu `Telebe` obyektini list-ə əlavə etsin. Eyni obyekt ikinci dəfə əlavə edilməsin.

3-cü hissə: Tələbələri göstərmək. `telebeleri_goster()` kursdakı tələbələrin ad və balını `for` dövrü ilə göstərsin. Boşdursa, `Kursda tələbə yoxdur` çap etsin.

4-cü hissə: Orta bal. `ortalama_bal()` bütün tələbələrin ballarını toplayıb tələbə sayına bölsün və nəticəni **return** etsin. Tələbə yoxdursa, `0` qaytarsın.

Yoxlama məlumatları:

Kurs	Müəllim	Tələbə	Bal
Python Backend Development	Elvin	Kamran	80
Python Backend Development	Elvin	Lalə	90

Gözlənilən orta bal: 85

İpucu: `Kurs` daxilində müəllim və tələbələrin ad/email məlumatlarını ayrıca təkrar yaratma. Artıq mövcud obyektləri saxla.

---

4. FINAL TAPŞIRIQ — Təlim mərkəzinin tamamlanması

Məqsəd: Bütün class və metodları eyni proqramda birləşdirmək. Aşağıdakı addımları ardıcıllıqla yerinə yetir.

Addım 1 — Müəllim yarat: `Muellim` obyekti: Elvin, `elvin@example.com`, ixtisas: Python Backend.

Addım 2 — Tələbələri yarat:

Ad	Email	İlkin bal
Kamran	kamran@example.com	80
Lalə	lale@example.com	90

Addım 3 — Balı yenilə: `bal_artir()` ilə Kamranın balına **10 xal əlavə et**. Yeni balı **90** olmalıdır.

Addım 4 — Kurs yarat: `Python Backend Development` kursunu yarat, Elvini müəllim təyin et. Kamran və Laləni kursa əlavə et.

Addım 5 — Tələbələri göstər: `telebeleri_goster()` vasitəsilə hər iki tələbənin adını və yenilənmiş balını çap et.

Addım 6 — Orta balı hesabla: `ortalama_bal()` metodunu çağır.

Tələbə	**Bal**
Kamran	90
Lalə	90
Orta bal	**90**

Addım 7 — Xətanı yoxla: Kamranın balını `150` etməyə çalış. `try/except` ilə xətanı tut. Yoxla ki, əvvəlki balı `90` olaraq qalıb.

Addım 8 — Class Method: `"Aysel;aysel@example.com"` mətnindən `Istifadeci.metnden_yarat()` vasitəsilə yeni obyekt yarat və məlumatlarını göstər.

Tamamlanma şərti: İstifadəçilər düzgün yaradılır, ballar yoxlanılır, kurs müəllim və tələbə obyektlərini saxlayır, orta bal `90` olur, yanlış bal qəbul edilmir.

Uğurlar!